

RACE Tutor AI

Classical Machine Learning System for Reading Comprehension Support

Abdullah Sajid Huzaifa Khalid
Roll No. 22i-2346 Roll No. 22i-2332

May 2026

Abstract

RACE Tutor AI is an interactive reading-comprehension tutoring application built on the RACE dataset. The system combines a classical machine-learning answer verification model with an extractive distractor and hint generator, then exposes the complete workflow through a Streamlit interface. Model A predicts the most likely answer option for a given passage and question. Model B generates plausible distractors and graduated hints from the source passage. The final Model A configuration uses one candidate example per answer option, binary bag-of-words features, bigrams, TF-IDF similarity, lexical overlap, negation features, and per-question relative option features. On the held-out test set, the final Model A reaches 37.81% four-option answer-selection accuracy.

Contents

1	Introduction	3
2	Dataset	3
2.1	Dataset Splits	3
3	Preprocessing	3
4	Model A: Answer Verification	4
4.1	Initial Framing	4
4.2	Final Candidate-Level Framing	4
4.3	Feature Engineering	4
4.4	Training Configuration	5
4.5	Model A Results	6
5	Model B: Distractor and Hint Generation	6
5.1	Distractor Generation	6
5.2	Hint Generation	7
6	Frontend Application	7
7	Verification	7

8	Limitations	8
9	Future Work	8
10	Conclusion	8

1 Introduction

Reading-comprehension systems are useful for education because they can help students practise passage analysis, answer selection, and evidence-based reasoning. This project implements a complete tutoring pipeline for the RACE dataset using transparent classical machine-learning methods. The goal is not to build a large neural model, but to create a reproducible system that can load RACE-style data, train an answer verifier, generate hints and distractors, and run inside an interactive application.

The project has three main components:

- **Model A:** an answer-verification and answer-selection model.
- **Model B:** a deterministic distractor and hint generator.
- **Frontend:** a Streamlit user interface for loading quizzes, answering questions, revealing hints, and viewing session analytics.

2 Dataset

The project uses the RACE dataset, a large-scale reading-comprehension dataset based on English examinations. Each example contains an article, a question, four answer options, and the correct option label. The local project expects the following flat CSV columns:

`id, article, question, A, B, C, D, answer`

The preprocessing code can also load the HuggingFace `ehovy/race` dataset and flatten each example from a nested `options` list into the explicit A, B, C, and D columns. Examples without exactly four options are ignored so that all downstream models receive a consistent four-choice structure.

2.1 Dataset Splits

Table 1 shows the cleaned split sizes used during the final training workflow. Cleaning removes invalid answers and rows with empty article or question text.

Table 1: Cleaned RACE split sizes used by the project.

Split	Rows	Candidate Rows	Answer A	Not A
Train	70,274	281,096	15,315	54,959
Validation	8,784	35,136	1,914	6,870
Test	8,787	35,148	1,915	6,872

3 Preprocessing

The preprocessing pipeline is implemented in `backend/preprocessing.py` and `backend/feature_engineering.py`. It performs the following steps:

1. Load local RACE CSV files or flatten HuggingFace RACE examples.
2. Keep only examples with four options.
3. Normalize text by lowercasing, removing punctuation, filling missing text, and normalizing whitespace.
4. Drop rows with invalid answer labels.
5. Drop rows with empty articles or questions.
6. Convert each question into candidate-level training examples.

For the final Model A, each original question is expanded into four candidate examples. Each candidate asks whether a particular option is the correct answer. This framing matches the final answer-selection task better than training on only option A.

4 Model A: Answer Verification

4.1 Initial Framing

The first Model A version used a binary option-A formulation:

Is option A correct?

The label was 1 when `answer == "A"` and 0 otherwise. This version was useful as a baseline and produced preprocessed artifacts in `data/processed/model_a`. However, it used only one option per question, which discarded information from options B, C, and D.

4.2 Final Candidate-Level Framing

The final model trains one example per candidate answer. For each RACE row, the system creates four examples:

`article + question + option A, article + question + option B, article +
question + option C, article + question + option D`

The label is 1 for the correct candidate and 0 for the other three candidates. At inference time, the model scores all four options and selects the option with the highest score.

4.3 Feature Engineering

The final feature set combines sparse text features and dense handcrafted features:

- Binary bag-of-words features from article, question, and candidate option text.
- Unigram and bigram features using `ngram_max = 2`.
- TF-IDF cosine similarity between the article and `question + option`.
- TF-IDF cosine similarity between the article and question.

- TF-IDF cosine similarity between the article and candidate option.
- Article length, question length, and option length.
- Question-article, option-article, and option-question lexical overlap.
- Option coverage ratio in the article.
- Negation flags for question and option text.
- Per-question relative option features, including option length ratio and overlap deltas against the sibling options.

The vectorizers and dense-feature scaler are fitted only on the training split. Validation and test data are transformed using the fitted training objects to avoid data leakage.

4.4 Training Configuration

The best final Model A was trained with logistic regression:

Table 2: Final Model A training configuration.

Setting	Value
Model	Logistic Regression
Training formulation	Four candidate examples per question
Class weight	None
Regularization parameter	$C = 0.3$
Maximum vocabulary size	50,000
Minimum document frequency	2
Maximum n-gram size	2
Saved artifact	models/model_a/logreg_candidate_rich.pkl

The exact command used for final training was:

```
.venv/bin/python -m backend.train_model_a \
  --train data/raw/train.csv \
  --val data/raw/val.csv \
  --test data/raw/test.csv \
  --model logreg_candidate \
  --class-weight none \
  --ngram-max 2 \
  --c 0.3 \
  --output-name logreg_candidate_rich
```

4.5 Model A Results

Table 3 shows the answer-selection improvement during development.

Table 3: Model A answer-selection accuracy progression.

Model	Validation Accuracy	Test Accuracy
Option-A binary model	0.3283	0.3214
Candidate model	0.3530	0.3570
Rich candidate model	0.3809	0.3781

The final binary candidate-level metrics are shown in Table 4. These metrics use the independent candidate labels. The answer-selection metric is more important for the application because the app ranks all four options for each question.

Table 4: Final Model A binary candidate-level metrics.

Split	Accuracy	Macro F1	Precision	Recall
Validation	0.7446	0.4671	0.4054	0.0459
Test	0.7444	0.4660	0.4002	0.0447

Table 5 shows the final four-option answer-selection metrics.

Table 5: Final Model A four-option answer-selection metrics.

Split	Answer Accuracy	Macro F1
Validation	0.3809	0.3798
Test	0.3781	0.3771

5 Model B: Distractor and Hint Generation

Model B is implemented in `backend/model_b.py`. It is a deterministic classical component rather than a trained model artifact. This design keeps the system lightweight and explainable.

5.1 Distractor Generation

The distractor generator extracts lightweight passage phrases without requiring external NLP models. It:

1. Splits the article into sentences.
2. Extracts content words while removing English stopwords.
3. Builds one- to four-word phrase candidates.

4. Filters out phrases that duplicate or contain the correct answer.
5. Scores candidates using passage frequency, question overlap, phrase length, and similarity to the correct answer.
6. Returns three distinct distractors.

5.2 Hint Generation

The hint generator ranks article sentences by overlap with the question and correct answer. It returns three hints:

1. A broad focus hint based on important question terms.
2. A supporting sentence with answer terms masked.
3. The strongest supporting sentence as direct evidence.

This gives the student a gradual path from general guidance to stronger evidence.

6 Frontend Application

The frontend is built in Streamlit and is implemented in `frontend/app.py`. It provides four tabs:

- **Article Input:** load a random RACE example or create a manual quiz.
- **Quiz View:** show the passage, question, answer options, and Model A prediction.
- **Hints:** reveal Model B hints one at a time and view generated distractors.
- **Analytics:** track attempts, accuracy, latency, and answer history for the current session.

The app uses `models/model_a/logreg_candidate_rich.pkl` as its default answer-verification model.

7 Verification

The completed project was verified through:

- Python byte-compilation of backend, frontend, and test modules.
- Backend smoke test covering dataset loading, quiz construction, Model A loading, Model A prediction, Model B distractor generation, and Model B hint generation.
- Streamlit server startup.
- HTTP response check from the running Streamlit app.

The Streamlit app was started successfully at:

<http://127.0.0.1:8501>

8 Limitations

Although the final model improves over the initial baselines, the system remains limited by its classical feature representation. RACE questions often require multi-sentence reasoning, inference, paraphrase recognition, and world knowledge. Bag-of-words and overlap features cannot fully capture these phenomena. Model B is also extractive and deterministic, so distractors can be surface-level and may not always match the semantic type of the correct answer.

9 Future Work

Future improvements include:

- Use stronger ranking models that optimize answer selection directly.
- Add semantic features from sentence embeddings or transformer models.
- Improve Model B with noun-phrase chunking and answer-type matching.
- Add persistent analytics and per-student progress tracking.
- Build a larger evaluation suite for generated distractor quality and hint usefulness.

10 Conclusion

RACE Tutor AI is now a complete running project. It loads and preprocesses RACE data, trains a classical answer-selection model, generates distractors and hints, and presents the workflow through a Streamlit interface. The strongest final Model A reaches 37.81% test answer-selection accuracy, improving over the option-A baseline and the first candidate-level model. Model B adds explainable educational support through passage-based distractors and graduated hints.

References

- [1] Guokun Lai, Qizhe Xie, Hanxiao Liu, Yiming Yang, and Eduard Hovy. *RACE: Large-scale ReAding Comprehension Dataset From Examinations*. EMNLP, 2017.
- [2] Fabian Pedregosa et al. *Scikit-learn: Machine Learning in Python*. Journal of Machine Learning Research, 2011.
- [3] Streamlit documentation. *Streamlit: A faster way to build and share data apps*. <https://streamlit.io/>.